Infrastructure-as-Code MLOps for Predictive Analytics

Anonymous Author(s)

Affiliations withheld for double-blind review

Abstract. Institutional predictive systems become difficult to reproduce when ingestion, evaluation, deployment, and reporting are implemented as disconnected steps. We present an Infrastructure-as-Code Machine Learning Operations (MLOps) workflow that binds these stages through a shared evidence contract: a deterministic adapter loads validated data into PostgreSQL, a blocked temporal evaluator compares forecasting candidates, and a schema-validated reporter generates traceable artifacts from one registered execution. Docker Compose provides the evaluated runtime, while Terraform declares the corresponding Azure topology. The case study uses 87,073 administrative publication records from 2015–2023, aggregated into nine annual observations. Across three chronological folds, eight candidates were compared using mean absolute percentage error (MAPE) and fold instability. Naive and simple exponential smoothing obtained the same score, and a parsimony rule selected Naive, which projected 15,229 records per year for 2024–2026 within an exploratory residual-calibrated 80% interval of 9,541.5–20,916.5. Host and container executions matched on all non-numeric fields, while 45 numeric differences satisfied the declared absolute-or-relative tolerance, although their complete payload hashes differed. The results support auditable traceability and tolerance-based numerical reproducibility within the evaluated case, but not byte-level identity or Azure runtime parity.

Keywords: reproducibility · auditable evidence · data provenance · temporal cross-validation · containerized workflows

1 Introduction

Machine learning (ML) systems now inform forecasting and institutional decisions [1], yet a prediction is only as trustworthy as the trail that documents how it was produced [2]. Machine Learning Operations (MLOps) builds that trail by drawing data ingestion, model development, deployment, monitoring, and governance into a single traceable lifecycle whose reliability in production is itself an active concern [3–6]. Reproducibility then rests on keeping four things intact: the identity of the dataset, the conditions under which it ran, the evidence behind model selection, and the artifacts produced along the way.

In practice these stages are often built and run in isolation. When they drift apart, the datasets, metrics, figures, and outputs produced in one environment

stop matching those produced in another. Prior taxonomies and data-pipeline studies map the lifecycle and document how schema, quality, and provenance failures propagate downstream [7–9], and data-centric perspectives treat data quality and governance as first-class pipeline concerns rather than afterthoughts [10].

Containerized and provenance-aware workflows have been proposed to preserve dependencies, runtime conditions, and processing records [7, 8, 11, 12]; recent work captures end-to-end provenance across an ML pipeline [13], and frameworks now support reproducible computational experiments across scientific domains [14]. Yet most of these efforts either target one platform capability or describe the lifecycle in the abstract, and what is still missing is a single contract that binds the identity of the data, the preprocessing decisions, the temporal evaluation, the artifacts these yield, and the comparison of results across environments.

This paper designs and evaluates such a contract. Built as Infrastructure-as-Code, the workflow connects deterministic data adaptation, PostgreSQL persistence, temporal evaluation, registered evidence, containerized execution, and deterministic reporting, while Terraform defines the corresponding Azure topology. Three research questions (RQs) frame the evaluation: do provenance records and scientific artifacts remain internally consistent (RQ1); are host and container executions numerically equivalent under a declared policy (RQ2); and which forecasting candidate is defensible for a nine-observation annual series under blocked temporal validation (RQ3). The case uses 87,073 administrative publication records from 2015–2023, aggregated into nine annual observations; therefore, the study examines traceability, evidence consistency, and tolerance-based reproducibility within a bounded setting rather than forecasting accuracy at large. We make no claim of a new algorithm, a production-grade platform, continuous monitoring, or Azure runtime parity. In Figure 1, controlled re-execution denotes a complete rerun using the same versioned source, code, configuration, and pinned dependencies; it is not an iterative model-retraining loop.

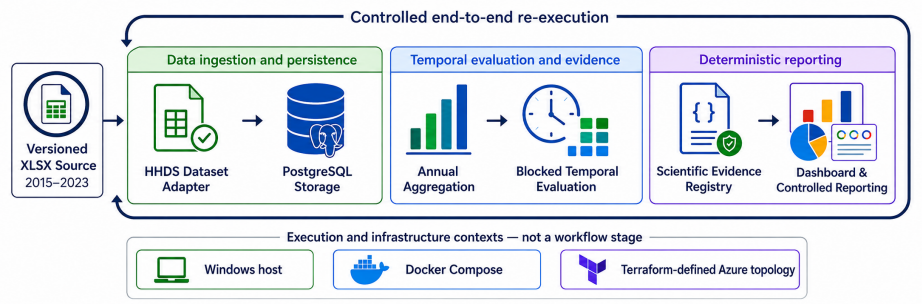

Fig. 1. Three-stage Infrastructure-as-Code MLOps workflow and its supporting execution contexts. Green, blue, and violet identify ingestion and persistence, evaluation and evidence, and deterministic reporting, respectively; the circular arrow denotes controlled end-to-end re-execution.

2 Methods

We developed the artifact under design-science guidance, maintaining transparency across its construction, evaluation, and communication [15, 16]. The process followed the design-science stages of identifying the problem, defining the solution objectives, designing and building the artifact, demonstrating its operation, and evaluating the resulting evidence.

2.1 Architecture, Ingestion, and Evidence Contract

The workflow contains three sequential stages implemented through six logical components: a dataset adapter, PostgreSQL persistence, a temporal evaluation engine, an evidence registry, a Streamlit interface, and a deterministic reporting service [7, 8]. Docker Compose provides the local realization evaluated in this study and standardizes its runtime configuration through pinned dependencies [11, 12]. Terraform defines the corresponding Azure realization, including Container Apps, networking, and PostgreSQL Flexible Server resources. The available Azure evidence is limited to the Terraform-managed resource topology and configuration; application execution, numerical parity, scalability, and runtime performance in Azure were not evaluated [17, 18]. Figure 2 relates the logical components to their local and cloud realizations.

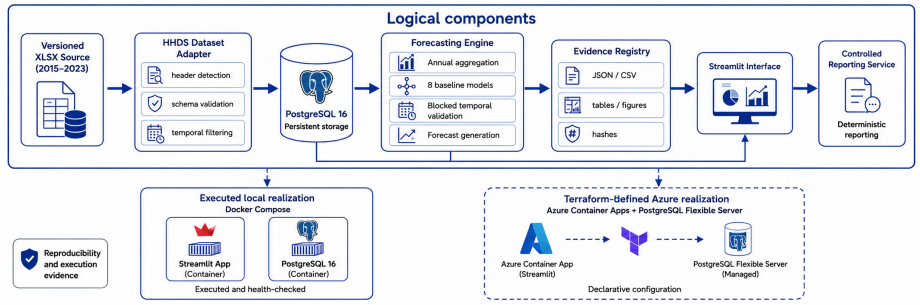

Fig. 2. Logical components and their execution and infrastructure realizations. Solid borders denote executed local components; dashed borders denote Terraform-defined Azure resources whose runtime parity was not evaluated.

We obtained the workbook from the official open-data repository of the Secretaría de Educación Superior, Ciencia, Tecnología e Innovación (SENESCYT) [19].. A deterministic adapter, HHDS (Heuristic Header Detection and Sanitization), prepares it in a fixed sequence: it scans the first 20 rows, keeps the candidate header whose overlap with the declared schema is highest, drops empty or undeclared fields, checks that every year falls within 2015 to 2023, normalizes the categorical values, records provenance, and writes the clean table into PostgreSQL 16 [9, 20]. The retained fields span four dimensions: temporal, institutional, publication, and disciplinary. Figure 3 lays out this pipeline step by step.

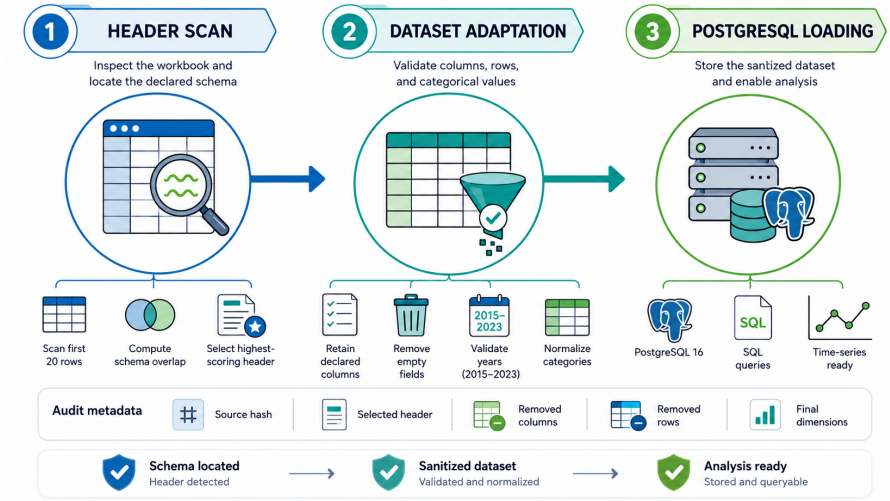

Fig. 3. Deterministic HHDS adaptation: header detection, schema validation, temporal filtering, normalization, provenance capture, and PostgreSQL loading.

Algorithm 1 states this procedure precisely, from header detection through sanitization to the registration of audit metadata.

Algorithm 1 Deterministic Header Detection and Dataset Sanitization

Require: Workbook D, declared schema $C^{\star}$, scan depth $N = 20$, valid years $[2015, 2023]$
Ensure: Sanitized dataset D_s and audit metadata
for $i = 1$ to N **do**
    Normalize candidate row r_i and compute its overlap with $C^{\star}$
end for
Select the earliest row among those with maximum schema overlap as the header
Remove preceding rows, unnamed columns, and empty rows
Retain declared fields and validate the year attribute
Normalize categorical values and enforce data types
Record source identity, selected header, removed fields, and row counts
Load D_s into PostgreSQL 16
return D_s and the registered audit metadata

Every analytical stage writes to one shared, schema-validated evidence registry. Its contract requires four groups of records: source identity and adaptation decisions; fold-level predictions, metrics, and selected model; forecasts and intervals; and generated artifacts with environment metadata, section hashes, and the complete scientific payload. The integrity check validates the required records, compares artifact fingerprints, and applies the declared host–container numerical policy. Because the tables, figures, and narrative read the same registered payload, the design reduces the risk of unnoticed inconsistencies without claiming byte-level identity. Figure 4 follows the evidence from generation and registration to integrity checking.

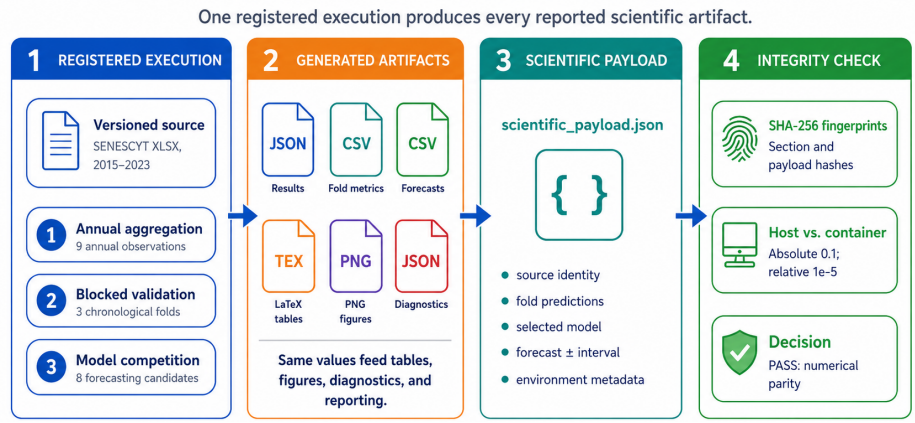

Fig. 4. Auditable evidence generation and integrity checking. Numerical equivalence uses the declared absolute-or-relative rule, while complete hashes preserve the distinction between numerical reproducibility and byte-level identity.

2.2 Temporal Evaluation and Forecasting

The target is the annual publication count. We compared eight candidates over three blocked folds that keep the years in chronological order [21–23]: Naive, simple exponential smoothing (SES), an automated exponential-smoothing state-space model (ETS), a linear trend, drift, the historical mean, a log-linear trend, and a Poisson generalized linear model (GLM). Equation (1) gives the fold-level mean absolute percentage error (MAPE) together with the instability-adjusted score that drives model selection.

$$\mathrm{MAPE}_{m,k} = \frac{100}{n_k} \sum_{t \in V_k} \left| \frac{y_t - \hat{y}_{m,t}}{y_t} \right|, \qquad S_\lambda(m) = \overline{\mathrm{MAPE}}_m + \lambda s_m, \tag{1}$$

where s_m is the sample standard deviation of the fold MAPE. In this study, S_λ is an explicit instability-penalized selection rule rather than a statistical-significance test. The primary ranking used $\lambda = 1$ and was rechecked at $\lambda \in \{0, 0.5, 1, 2\}$; ties in the registered score were resolved by parsimony.

For the chosen model, six absolute out-of-fold residuals define an exploratory, residual-calibrated interval, written in Eq. (2).

$$PI_{h,0.8} = [\max(0, \hat{y}_h - q_{0.8}),\ \hat{y}_h + q_{0.8}], \tag{2}$$

where $q_{0.8}$ is the empirical residual quantile. The idea follows conformal prediction, though a calibration set this small and this dependent cannot promise population-level coverage [24, 25]. Figure 5 brings the validation, the candidate comparison, the instability-adjusted ranking, and the final choice of Naive into one view.

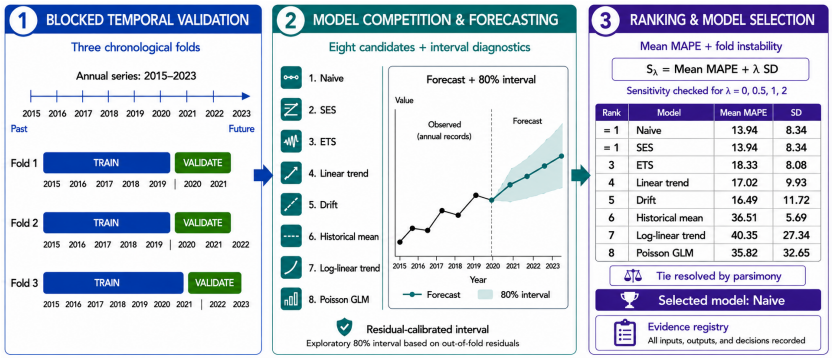

Fig. 5. Blocked temporal validation and instability-adjusted comparison of eight forecasting candidates. Naive was selected by the stated parsimony rule after obtaining the same registered score as SES.

Algorithm 2 sets out the full computation, from temporal evaluation and forecasting to the registration of evidence.

Algorithm 2 Blocked Temporal Evaluation and Evidence Registration

Require: Annual series Y, candidate set $\mathcal{M}$, three chronological folds, horizon $H = \{2024, 2025, 2026\}$
Ensure: Selected model, forecasts, interval, and registered scientific payload

for each candidate $m \in \mathcal{M}$ **do**
  for each chronological fold k **do**
    Fit m using only observations preceding validation block V_k
    Predict V_k and compute $\text{MAPE}_{m,k}$
    Store fold predictions and absolute residuals
  end for
  Compute mean MAPE, fold SD, and the primary score $S_1(m)$
end for
Select the minimum-score candidate and resolve exact ties by parsimony
Refit the selected model using the complete annual series
Generate forecasts for H and the exploratory residual-calibrated interval
Register folds, metrics, forecasts, figures, diagnostics, and hashes
return the registered scientific payload

2.3 Reporting and Reproducibility Protocol

The reporting path we evaluated is deterministic and offline: a schema-validated template renders the narrative, tables, and figures straight from the registered payload. A GPT-4o-mini route exists as an option but was left switched off, so we say nothing here about the factuality, usefulness, or stability of large language models [26].

We compared host and container runs using complete and section-level Secure Hash Algorithm 256-bit (SHA-256) fingerprints to detect byte-level divergence in the registered outputs. Non-numeric values had to match character for character.

Two numeric values a and b were treated as equivalent when they satisfied either branch of Eq. (3); the branches are joined by a logical OR.

$$|a-b| \le 0.1 \quad \vee \quad \frac{|a-b|}{\max(|a|,|b|)} \le 10^{-5}. \tag{3}$$

The policy therefore accepts an absolute difference of at most 0.1 or a relative difference of at most 10^{-5}. The validated Linux environment, executed through Windows Subsystem for Linux 2 (WSL2), exposed eight logical CPUs and approximately 15.52 GB of memory, with no GPU access. Its Python 3.14.6 environment was frozen in `requirements.lock`; key versions included pandas 3.0.3, NumPy 2.5.1, scikit-learn 1.9.0, statsmodels 0.14.6, SQLAlchemy 2.0.51, and Streamlit 1.59.2. The Windows host used Python 3.14.5. Because the Python patch levels differed, the experiment evaluates tolerance-based numerical reproducibility across controlled, near-identical stacks rather than bitwise reproducibility under an identical runtime.

3 Results

Every result below comes from one frozen host execution and its containerized replica. The comparison held the SENESCYT workbook, pipeline code, configuration, comparison policy, and selected model fixed, while the recorded environments differed in operating system and Python patch level. The observed divergences are therefore interpreted as runtime-associated differences rather than differences caused by distinct inputs or analytical procedures. This design provides a controlled comparison of numerical reproducibility, although it does not establish bitwise equivalence. The host execution serves as the reference, and the reported values, fingerprints, and intervals are those stored in its evidence registry rather than quantities recomputed for presentation.

3.1 Execution and Cloud Configuration Evidence

Ingestion processed the 87,073 records and distinguished 16,599 raw journal-title strings in 16.44 s, about 5,296 records per second for this one workbook on this one machine; this is a single measurement, not a scalability figure. Figure 6 records this extract–transform–load (ETL) run.

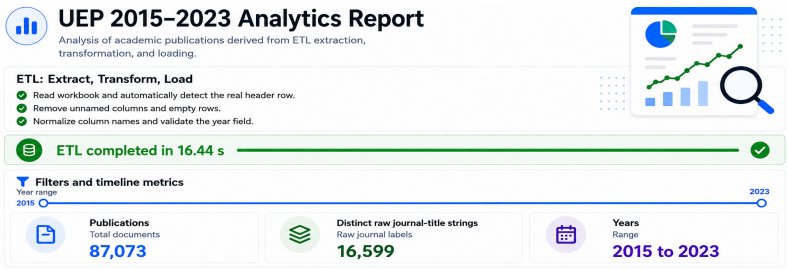

Fig. 6. Extract–transform–load run: 87,073 records and 16,599 distinct raw journal-title strings processed in 16.44 s.

The dashboard records the deterministic gates applied before the counts were produced: header detection, removal of unnamed columns and empty rows, column-name normalization, and year validation. The 87,073 records therefore constitute the post-sanitization reference count used in the host–container comparison rather than an unverified raw total. The 16,599 distinct raw journal-title strings describe the string-level heterogeneity handled by the adapter and provide provenance context for later normalization; they are not presented as a performance or scalability result.

Terraform defines the Azure application, its container environment, networking, and managed PostgreSQL instance. Figure 7 records the Terraform-managed resource group available for this case. The evidence supports correspondence between the declared configuration and the listed resources, but it does not include application startup, health responses, remote database queries, cloud-generated payloads, execution times, or a host Azure comparison. Accordingly, the figure documents configuration and provisioning evidence, not application execution or runtime parity.

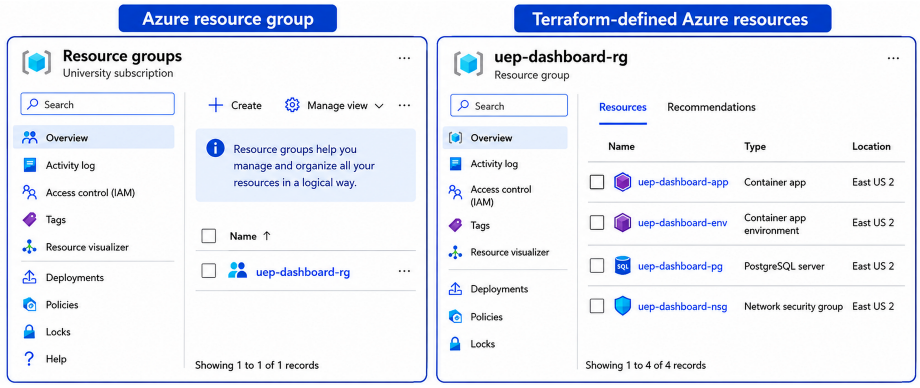

Fig. 7. Terraform-managed Azure resources recorded for the case study. Application execution and runtime parity were not evaluated.

The four recorded resources a Container App, a Container App Environment, a PostgreSQL server, and a Network Security Group correspond to the application, database, and network roles of the evaluated Docker Compose stack. Because Terraform state enumerates these resources, reapplying the same configuration is intended to reproduce an equivalent topology, subject to provider versions, state, credentials, regional availability, and cloud-platform constraints. The claim is therefore limited to configuration traceability; application behavior and runtime security enforcement remain untested.

3.2 Host Container Reproducibility

The host ran Windows 11 with Python 3.14.5, whereas the WSL2 container ran Linux with Python 3.14.6. Their complete payload hashes differed, and the comparison localized 45 floating-point differences within the forecasting section;

every recorded non-numeric field matched exactly. Table 1 summarizes the comparison.

Table 1. Host container reproducibility evidence.

Evidence	Recorded result
Payload SHA-256	Host: `ed7198a8bf5d...` Container: `a072859cc944...`
Byte-level parity	No; complete payload fingerprints differ
Numerical comparison	45 differences; max. absolute 0.0710614; max. relative 3.9943×10^{-5}
Policy and outcome	$\lvert a-b\rvert \leq 0.1$ **OR** relative difference $\leq 10^{-5}$; passed. Cases exceeding the relative threshold satisfied the absolute branch
Registered output	Naive selected in both; registered forecasts and intervals satisfied the declared rule

The reproducibility we can claim, then, is numerical and policy-bound: not byte-level identity, and not Azure runtime parity.

3.3 Descriptive and Forecasting Results

Annual counts climbed from 3,936 in 2015 to 15,229 in 2023 (3,936, 5,485, 8,611, 9,103, 9,518, 10,326, 11,088, 13,777, and 15,229), though these are administrative records rather than a direct gauge of scientific output. Figure 8 shows the leading journal labels under English display names, while the counts stay tied to the original raw strings.

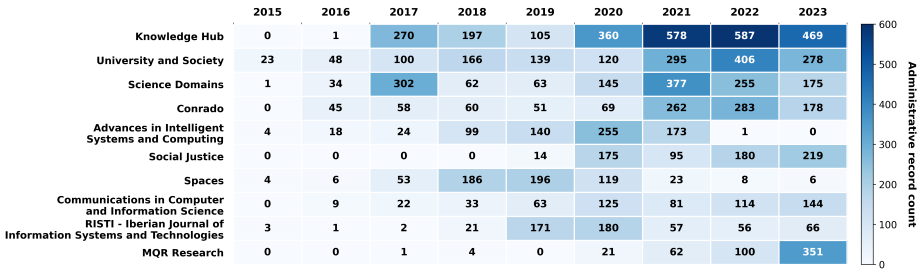

Fig. 8. Annual counts for the ten journal labels with the highest accumulated record totals during 2015–2023. English display labels are used only for presentation; the registered counts remain linked to the original source strings.

Naive and SES produced the same registered mean MAPE of 13.94, fold standard deviation of 8.34, and $S_1 = 22.28$; the stated parsimony rule therefore selected Naive. The Poisson GLM achieved 3.5% MAPE in the final fold but showed substantially larger errors in the earlier folds, so its instability-adjusted score did not support selection. Varying λ did not change the Naive–SES tie.

Naive therefore carries the 2023 level forward, projecting 15,229 records for each of 2024, 2025, and 2026, within an exploratory 80% interval of 9,541.5–

20,916.5 (a residual radius of 5,687.5 records). Figure 9 is drawn straight from the registered forecast entries, not recomputed in the reporting layer.

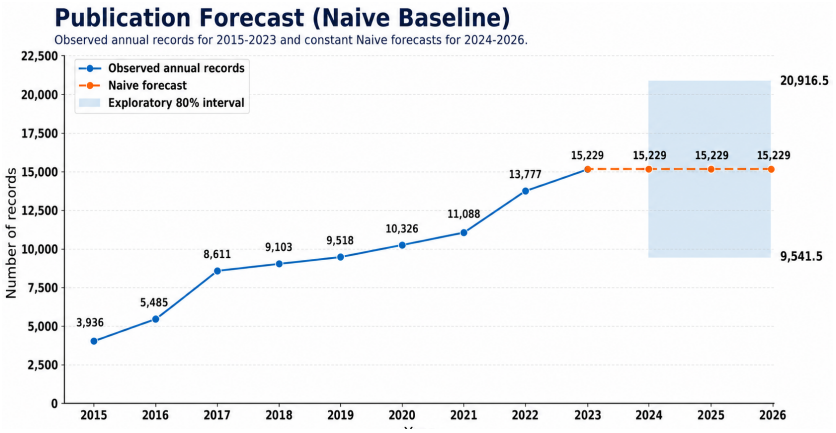

Fig. 9. Observed records and Naive forecasts with an exploratory 80% interval.

3.4 Deterministic Reporting

The deterministic reporter reused the registered forecasts, intervals, and diagnostics that supplied the remaining scientific artifacts. Because the report reads the registered payload rather than recomputing analytical values, this design reduces the risk of divergence between the analysis and its presentation. The optional GPT-4o-mini route was configured but not executed or evaluated; it therefore contributes no reported result and remains outside the evidence assessed in this study.

4 Discussion

Within the boundaries of this case study, the results provide supporting evidence for the three research questions. For RQ1, the shared registry generated the reported tables, figures, forecasts, and diagnostics from one registered payload, supporting internal consistency, although deliberate corruption or failure-injection tests were not performed. For RQ2, every non-numeric field matched exactly, while the 45 numeric differences satisfied the declared tolerance policy, supporting policy-bounded numerical equivalence rather than byte-level identity. For RQ3, Naive and SES obtained the same registered score, and the stated parsimony rule selected Naive. These findings concern traceability and controlled execution within one dataset and one host–container pair; they do not establish broad forecasting superiority or production-scale MLOps performance. Relative to MLOps taxonomies, lifecycle reviews, and data-centric accounts [4, 7, 8, 20, 10], the contribution is complementary: the evidence contract connects identity, preprocessing, fold predictions, metrics, forecasts, figures, reports, and hashes

through one registered execution, thereby reducing the risk of unnoticed cross-artifact inconsistencies.

This places the work alongside the container- and provenance-based reproducibility literature [11, 12], but with a different emphasis. Containers standardize software environments [11], while provenance-aware engines record execution histories [12], and recent systems extend this capture across complete ML pipelines [13]. The host–container comparison indicates that such preservation can remain numerical rather than bitwise: the near-identical stacks based on Python 3.14.5 and 3.14.6 produced 45 small floating-point differences. This gap between code availability and identical results is among the barriers documented in surveys of ML reproducibility [2] and addressed by cross-domain reproducibility frameworks [14]. Consistent with studies that connect downstream errors with schema and provenance failures [9], the deterministic adapter and evidence registry provide points at which inconsistencies can be detected before reaching the reported artifacts. Unlike cloud-scaling and platform studies that evaluate runtime behavior [17, 18], the Azure portion of this work supports configuration correspondence rather than execution parity.

On the forecasting side, the tie between simple baselines and more parameterized models is what the small-sample and temporal-validation literature would anticipate. With nine annual points and three blocked folds, blocked temporal validation is the defensible protocol [21, 22], and results on model selection under temporal distribution shift caution against reading a single strong fold as durable evidence [23]; the Poisson GLM, strong in the final fold yet unstable earlier, is a clear instance. The exploratory interval follows conformal principles [24, 25], but we deliberately stop short of a coverage guarantee because the calibration set is small and non-exchangeable, precisely the regime those works flag. Within this case, the simple baselines were therefore more defensible than the parameterized alternatives. The wide interval reflects the uncertainty imposed by the short series and should not be interpreted as a general performance property of the workflow.

Reproducibility in this setting is evaluated under an explicit tolerance rather than as exact binary identity, and the mismatched Python patch versions remain a limitation that a stricter replication should remove. Read against work on trustworthy and well-governed AI pipelines [26, 6, 1], the evidence contract operationalizes a narrow but auditable part of that agenda: it records the execution evidence and supports later recomputation when the code, source data, configuration, and dependency lock are available. It does not yet address continuous training, drift, reliability, security, cost, or cloud scalability.

Construct validity is limited by the outcome variable because an administrative record count is only a proxy for scientific productivity. Statistical-conclusion validity rests on nine observations, three folds, and six calibration residuals; consequently, neither the ranking differences nor the interval coverage should be generalized beyond this case. Internal validity depends on the frozen adapter, fixed fold definitions, pinned dependencies, and evidence registry, while the host–container comparison permits limited numerical variation by design.

External validity is restricted to one dataset and one host–container pair; Azure was assessed at the configuration and resource-provisioning level only, and the language-model route remained disabled.

5 Conclusions

The Infrastructure-as-Code workflow organized data adaptation, persistence, temporal evaluation, evidence registration, containerized execution, cloud configuration, and deterministic reporting around one registered execution designed for re-execution and audit. Following design-science guidance on transparent and communicable artifacts [15, 16], the evidence contract constitutes the central contribution. Ingestion processed the evaluated workbook in 16.44 s, while the reproducibility comparison found exact agreement for all non-numeric fields and tolerance-level agreement for the 45 numeric differences, despite distinct complete payload hashes.

For forecasting, Naive and SES obtained the same registered score ($S_1 = 22.28$), and the stated parsimony rule selected Naive. This result is relevant primarily as evidence that model selection, forecasts, intervals, and reporting artifacts were derived from the same registered payload, rather than as evidence of forecasting superiority. The study therefore extends container- and provenance-oriented reproducibility [11, 12] through an explicit evidence contract, within the limitations of one dataset, one host–container pair, and a non-executed Azure application workload. Future work should evaluate the complete Azure runtime, repeat the study in an independent domain, align host and container interpreter versions, and incorporate continuous training and drift monitoring.

Acknowledgements. Acknowledgements are withheld for double-blind review and will be added in the camera-ready version.

References

1. Abdalilah Alhalangy. Operationalizing accountable ai through traceable governance architecture for institutional decision support. *Information*, 17(7):694, 2026. https://doi.org/10.3390/info17070694.
2. Harald Semmelrock, Tony Ross-Hellauer, Simone Kopeinik, Dieter Theiler, Armin Haberl, Stefan Thalmann, and Dominik Kowald. Reproducibility in machine-learning-based research: Overview, barriers, and drivers. *AI Magazine*, 46(2):e70002, 2025. https://doi.org/10.1002/aaai.70002.
3. Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. https://doi.org/10.1109/ACCESS.2023.3262138.
4. Josu Diaz-de Arcaya, Ana I. Torre-Bastida, Gorka Zárate, Raúl Miñón, and Aitor Almeida. A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey. *ACM Computing Surveys*, 56(4):1–30, 2024. https://doi.org/10.1145/3625289.

5. Beyza Eken, Samodha Pallewatta, Nguyen Khoi Tran, Ayse Tosun, and Muhammad Ali Babar. A multivocal review of MLOps practices, challenges and open issues. *ACM Computing Surveys*, 58(2):1–35, 2025. https://doi.org/10.1145/3747346.
6. Firas Bayram and Bestoun S. Ahmed. Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach. *ACM Computing Surveys*, 57(5):1–35, 2025. https://doi.org/10.1145/3708497.
7. Matteo Testi, Matteo Ballabio, Emanuele Frontoni, Giulio Iannello, Sara Moccia, Paolo Soda, and Gennaro Vessio. MLOps: A taxonomy and a methodology. *IEEE Access*, 10:63606–63618, 2022. https://doi.org/10.1109/ACCESS.2022.3181730.
8. Faezeh Amou Najafabadi, Justus Bogner, Ilias Gerostathopoulos, and Patricia Lago. An architectural perspective on MLOps: Structures, processes, tools, and stakeholders. *Information and Software Technology*, 193:108029, 2026. https://doi.org/10.1016/j.infsof.2026.108029.
9. Harald Foidl, Valentina Golendukhina, Rudolf Ramler, and Michael Felderer. Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software*, 207:111855, 2024. https://doi.org/10.1016/j.jss.2023.111855.
10. Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025. https://doi.org/10.1145/3711118.
11. David Moreau, Kristina Wiebels, and Carl Boettiger. Containers for computational reproducibility. *Nature Reviews Methods Primers*, 3:50, 2023. https://doi.org/10.1038/s43586-023-00236-9.
12. Sebastiaan P. Huber. Automated reproducible workflows and data provenance with AiiDA. *Nature Reviews Physics*, 4:431, 2022. https://doi.org/10.1038/s42254-022-00463-1.
13. Marius Schlegel and Kai-Uwe Sattler. Capturing end-to-end provenance for machine learning pipelines. *Information Systems*, 132:102495, 2025. https://doi.org/10.1016/j.is.2024.102495.
14. Lázaro Costa, Susana Barbosa, and Jácome Cunha. A framework for supporting the reproducibility of computational experiments in multiple scientific domains. *Future Generation Computer Systems*, 174:107924, 2026. https://doi.org/10.1016/j.future.2025.107924.
15. Petrus M. J. Delport, Rossouw von Solms, and Mariana Gerber. Methodological guidelines for design science research. *Procedia Computer Science*, 237:195–203, 2024. https://doi.org/10.1016/j.procs.2024.05.096.
16. Alan R. Hevner, Jeffrey Parsons, Alfred B. Brendel, Roman Lukyanenko, Verena Tiefenbeck, Monica Chiarini Tremblay, and Jan vom Brocke. Transparency in design science research. *Decision Support Systems*, 182:114236, 2024. https://doi.org/10.1016/j.dss.2024.114236.
17. G. Ramesh, T. Vaikunta Pai, Ramona Birau, Karthik K. Poojary, Abhay, Aishwarya R. Shingad, N. Sowjanya, Virgil Popescu, Adrian T. Mitroi, Roxana M. Nioata, and K. M. Kiran Raj. A comprehensive review on scaling machine learning workflows using cloud technologies and DevOps. *IEEE Access*, 13:148559–148594, 2025. https://doi.org/10.1109/ACCESS.2025.3599281.
18. Lisana Berberi, Valentin Kozlov, Giang Nguyen, Judith Sáinz-Pardo Díaz, Amanda Calatrava, Germán Moltó, Viet Tran, and Álvaro López García. Machine learning operations landscape: Platforms and tools. *Artificial Intelligence Review*, 58:167, 2025. https://doi.org/10.1007/s10462-025-11164-3.

19. SENESCYT. Base de datos de artículos publicados uep en revistas indexadas — conjunto de datos — datos abiertos ecuador. https://datosabiertos.gob.ec/dataset/base-de-datos-de-articulos-publicados-uep-en-revistas-indexadas, 2025. Accessed on 10 January 2026.
20. Adrian P. Woźniak, Mateusz Milczarek, and Joanna Woźniak. MLOps components, tools, process, and metrics: A systematic literature review. *IEEE Access*, 13:22166–22175, 2025. https://doi.org/10.1109/ACCESS.2025.3534990.
21. Ai Deng. Time series cross validation: A theoretical result and finite sample performance. *Economics Letters*, 233:111369, 2023. https://doi.org/10.1016/j.econlet.2023.111369.
22. Ayan Chatterjee, Bestoun S. Ahmed, Erik Hallin, and Anton Engman. Testing of machine learning models with limited samples: An industrial vacuum pumping application. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1280–1290, Singapore, 2022. https://doi.org/10.1145/3540250.3558943.
23. Elise Han, Chengpiao Huang, and Kaizheng Wang. Model assessment and selection under temporal distribution shift. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 17374–17392, 2024. https://proceedings.mlr.press/v235/han24b.html.
24. Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. https://doi.org/10.1561/2200000101.
25. Roberto I. Oliveira, Paulo Orenstein, Thiago Ramos, and João V. Romano. Split conformal prediction and non-exchangeable data. *Journal of Machine Learning Research*, 25:1–38, 2024. Article 225.
26. Talal Ashraf Butt, Muhammad Iqbal, and Noor Arshad. From policy to pipeline: A governance framework for ai development and operations pipelines. *IEEE Access*, 14:1373–1397, 2026. https://doi.org/10.1109/ACCESS.2025.3647479.